arc42

Software Architektur Dokumentation

Caruso - Promptify

Projektname	*Caruso Dashboard*
Version	*0.1*
Datum	*8.4.2026*
Status	*Entwurf*
Autoren	*Tomas Parada, Tobias Gladhorn i.A. Promptify*
Reviewers	*Adrian Gareis, Luis Däuber, Ioana Purdila*

Inhaltsverzeichnis

0. Über dieses Dokument

1. Einführung und Ziele

1.1 Aufgabenstellung

1.2 Qualitätsziele

1.3 Stakeholder

2. Randbedingungen

2.1 Technische Randbedingungen

2.2 Organisatorische Randbedingungen

2.3 Konventionen

3. Kontextabgrenzung

3.1 Fachlicher Kontext

3.2 Technischer Kontext

4. Lösungsstrategie

5. Bausteinsicht

5.1 Whitebox Gesamtsystem

5.2 Blackbox Beschreibungen

5.2.1 Baustein A

6. Laufzeitsicht

6.1 Szenario 1

6.2 Szenario 2

7. Verteilungssicht

7.1 Infrastruktur

7.2 Zuordnung Bausteine zu Infrastruktur

8. Querschnittliche Konzepte

8.1 Sicherheitskonzept

8.2 Logging & Monitoring

8.3 Fehlerbehandlung

8.4 Konfigurationsmanagement

8.5 Datenmigration

9. Architekturentscheidungen (ADRs)

9.1 ADR-001

9.2 ADR-002

10. Qualitätsanforderungen

10.1 Qualitätsszenarien

11. Risiken und technische Schulden

11.1 Risiken

11.2 Technische Schulden

12. Glossar

0. Über dieses Dokument

Dieses Dokument folgt dem arc42-Template (Version 8.x) für die Dokumentation von Software-Architekturen.

arc42 ist ein pragmatisches Template für Architekturdokumentation, entwickelt von Gernot Starke und Peter Hruschka.

Es strukturiert Architekturinformationen in 12 Abschnitte.

1. Einführung und Ziele

Dieser Abschnitt beschreibt die wesentlichen Anforderungen und treibenden Kräfte, die bei der Umsetzung der Softwarearchitektur berücksichtigt werden müssen.

1.1 Aufgabenstellung

Die Aufgabe ist es, eine Tabelle mit mehr als 400 Einträgen für den Besucher der Caruso Website so neu zu strukturieren, dass Besucher übersichtlicher finden, was sie brauchen und überschauen können, welche Charakteristika vorliegen.
Direkte Aufgabenstellung des kunden:

Visualization of the CARUSO Data Catalog

The data catalog contains **more than 400 data items** (and growing)

- Data items are displayed in a flat table/list
- Basic feature like sorting, filtering, searching, exporting to .csv are available
- Allows to request a subscription

But...

- How can our customers better understand what kind of data is available?
- How can our customers better find what they need?
- How can our customers better learn about the characteristics of the data?

1.2 Qualitätsziele

Qualitätsmerkmal	Priorität	Beschreibung / Szenario
Performance	hoch	Dem User ein interaktives Erlebnis bieten
Skalierbarkeit	mittel	Es sollen weiter Werte hinzugefügt werden können
Sicherheit	mittel	Firmeninterne Daten werden DSGVO konform behandeln
Wartbarkeit	hoch	Entwickler den kunden können den Code sehr gut durchblicken
Verfügbarkeit	hoch	Die Visualisation soll immer erreichbar sein wenn die Website erreichbar ist

1.3 Stakeholder

Stakeholder	Rolle	Erwartungen	Verfügbarkeit
Auftraggeber	*Zielsetzung*	*Lauffähiges Prototyp und neue Ideenfindung.*	*Werktags, zwischen 9:00 - 18:00 Uhr, via Teams.*
Betreuer	*Evaluation und Betreuung des Projektes.*	*Kohärente und geplante Erfüllung der vom Team vorgegebenen Ziele.*	-
Designer	*Gestaltung der UX- und UI der Webseite und Projekt zusammen mit dem Team.*	*Kohäsives und ordentliches Arbeiten, Fortgeschrittene Projekt für das eigene Portfolio.*	-
Team Mitglieder	*Führung und Entwicklung des Projektes.*	-	-

2. Randbedingungen

💡 *Randbedingungen schränken den Lösungsraum ein und gelten für alle Entscheidungen.*

2.1 Technische Randbedingungen

Programmiersprache	*[z.B. Java 17, Python 3.11]!!!später*
Laufzeitumgebung	*Docker*
Datenbank	*.csv Datei, Supabase*

2.2 Organisatorische Randbedingungen

Team	*[Teamgröße: 5, Standort: 5*
Zeitplan	*[Meilensteine, Release-Datum]*
Vorgehensmodell	*Scrum*

2.3 Konventionen

[z.B. Coding Guidelines, Namenskonventionen, Dokumentationsstandards]

3. Kontextabgrenzung

💡 *Zeigt das System in seiner Umgebung und alle externen Schnittstellen.*

3.1 Fachlicher Kontext

[Kontextdiagramm einfügen – Systemname im Mittelpunkt, externe Akteure/Systeme als Kästen, Pfeile beschriften]

Kommunikationspartner	Beschreibung (Input / Output)
[Externer Nutzer]	*[Eingaben und Ausgaben beschreiben]*

3.2 Technischer Kontext

[Technische Schnittstellen: Protokolle, Kanäle, Hardware]

4. Lösungsstrategie

💡 *Kurze Zusammenfassung der fundamentalen Architekturentscheidungen.*

Architekturansatz	*[z.B. Microservices, Monolith, Event-driven]*
Technologieentscheidungen	*[Wichtigste Tech-Stack-Entscheidungen]*
Organisationsstruktur	*[z.B. Feature-Teams, Layered]*
Qualitätserreichung	*[Wie werden Qualitätsziele aus Abschnitt 1 erreicht?]*

5. Bausteinsicht

💡 *Statische Zerlegung des Systems in Bausteine (Module, Komponenten, Subsysteme).*

5.1 Whitebox Gesamtsystem

[Übersichtsdiagramm des Gesamtsystems einfügen]

Baustein	Verantwortlichkeit / Zweck
[Baustein A]	*[Beschreibung]*
[Baustein B]	*[Beschreibung]*
[Baustein C]	*[Beschreibung]*

5.2 Blackbox Beschreibungen

5.2.1 [Baustein A]

Zweck	*[Aufgabe des Bausteins]*
Schnittstellen	*[Bereitgestellte Schnittstellen]*
Qualitätsmerkmale	*[Spezifische Qualitätsanforderungen]*
Offene Punkte	*[Bekannte Probleme oder Entscheidungen]*

6. Laufzeitsicht

💡 *Beschreibt das Verhalten der Bausteine zur Laufzeit durch Szenarien / Sequenzdiagramme.*

6.1 Szenario: [Name des Szenarios]

[Sequenzdiagramm oder Beschreibung des Ablaufs einfügen]

Auslöser	*[Was startet das Szenario?]*
Ablauf	*[Beschreibung der Schritte]*
Ergebnis	*[Was ist das Ergebnis?]*

6.2 Szenario: [Weiteres Szenario]

[Weitere Szenarien nach demselben Muster ergänzen]

7. Verteilungssicht

> 💡 *Zeigt die technische Infrastruktur und die Zuordnung von Bausteinen zu Infrastrukturelementen.*

7.1 Infrastruktur

[Deployment-Diagramm einfügen]

Umgebung	*[Prod / Test / Dev]*
Plattform	*[z.B. AWS, Azure, On-Premise]*
Redundanz	*[HA-Konzept, Failover]*

7.2 Zuordnung Bausteine zu Infrastruktur

[Baustein A]	*[Läuft auf: Container / Server / Serverless]*
[Baustein B]	*[Läuft auf: ...]*

8. Querschnittliche Konzepte

> 💡 *Übergreifende Prinzipien und Lösungsansätze, die in vielen Teilen der Architektur relevant sind.*

8.1 Sicherheitskonzept (kann weggelassen werden)

Secrets Management	*[z.B. Vault, AWS Secrets Manager]*

8.2 Logging & Monitoring

Logging-Framework	*[z.B. ELK Stack, Loki]*
Monitoring	*[z.B. Prometheus, Grafana, Datadog]*
Alerting	*[z.B. PagerDuty, Alertmanager]*
Tracing	*[z.B. OpenTelemetry, Jaeger]*

8.3 Fehlerbehandlung

[Strategie für Fehlerbehandlung, Retry-Logik, Circuit Breaker etc.]

8.4 Konfigurationsmanagement

[Wie wird Konfiguration verwaltet? z.B. Environment Variables, Config Maps, Feature Flags]

8.5 Datenmigration

[Strategie für DB-Migrationen, z.B. Flyway, Liquibase]

9. Architekturentscheidungen

💡 *Wichtige Architekturentscheidungen als ADRs (Architecture Decision Records) dokumentieren.*

9.1 ADR-001: [Titel der Entscheidung]

Status	*Vorgeschlagen / Akzeptiert / Abgelehnt / Überholt*
Kontext	*[Warum war diese Entscheidung notwendig?]*
Entscheidung	*[Was wurde entschieden?]*
Alternativen	*[Welche Alternativen wurden betrachtet?]*
Konsequenzen	*[Positive und negative Konsequenzen]*
Datum	*[TT.MM.JJJJ]*

9.2 ADR-002: [Titel der Entscheidung]

Status	
Kontext	
Entscheidung	
Alternativen	
Konsequenzen	
Datum	

10. Qualitätsanforderungen

💡 *Konkretisierung der Qualitätsziele aus Abschnitt 1 durch messbare Szenarien.*

10.1 Qualitätsszenarien

ID	Qualitätsmerkmal	Szenario	Maßnahme	Metrik
QS-01	*Performance*	*1000 gleichzeitige User*	*Caching, Load Balancing*	*Antwortzeit < 500ms*

11. Risiken und technische Schulden

💡 *Bekannte Risiken und technische Schulden transparent machen.*

11.1 Risiken

Risiko	Eintrittswahrsch.	Auswirkung	Maßnahme
[Risikobezeichnung]	*mittel*	*hoch*	*[Gegenmaßnahme]*

11.2 Technische Schulden

Technische Schuld	Auswirkung	Priorität / Geplante Behebung
[Beschreibung]	*[Betroffene Bereiche]*	*mittel / Q2 2026*

12. Glossar

💡 *Wichtige fachliche und technische Begriffe des Projekts.*

Begriff	Definition
ADR	*[Definition eintragen]*
API	*[Definition eintragen]*
arc42	*[Definition eintragen]*